| | |
|---|---|
| ***Document Number:*** <QMS Ref. No.> | ***Copy Number:*** <copy number> |

**TATA CONSULTANCY SERVICES**

**Trivandrum**

# VILP – Struts2 – Part III

## Content Manual

Version 1.2

**June 2014**

TATA CONSULTANCY SERVICES

A Division of **Tata Sons Limited**

# DOCUMENT RELEASE NOTICE

Notice No.
Client
Project
Document details

| Name | Version No. | Author | Description |
|---|---|---|---|
| **Struts2 – Part III** | 1 | Priyanka Sabulal | |

Revision details:

| Action taken (add/del/chg) | Preceding page No. | New page No. | Revision description |
|---|---|---|---|
| | | | |

Change Register serial numbers covered:
The documents or revised pages are subject to document control.
Please keep them up-to-date using the release notices from the distributor of the document.
These are confidential documents. Unauthorized access or copying is prohibited.

Approved by : _____ Date: _____

Authorised by: _____ Date: _____

# DOCUMENT REVISION LIST

Client
Project
Document Name
Release Notice Reference (for release)

| Rev. No. | Revision date | Revision description | Page No. | Prev page No. | Action taken | Addenda /New page | Release Notice reference |
|---|---|---|---|---|---|---|---|
| | | | | | | | |

# Table of Contents

**COURSE 3 STRUTS2 – PART III ..... 1**

3.1 Objective ..... 1
3.2 Course Content ..... 1
3.2.1 More about Struts.xml ..... 1
3.2.2 Exploring Struts.xml ..... 1
3.2.2.1 More about Package Element ..... 2
3.2.2.1.1 Attributes of Package Element ..... 3
3.2.2.1.1.1 name ..... 3
3.2.2.1.1.2 Namespace ..... 3
3.2.2.1.1.2.1 Default Namespace ..... 3
3.2.2.1.1.2.2 Root Namespace ..... 3
3.2.2.1.2 Action Element ..... 4
3.2.2.1.2.1 Attributes of action element ..... 5
3.2.2.1.3 Result Element ..... 5
3.2.2.1.3.1 Attributes of result element ..... 5
3.2.2.1.3.2 Global Results ..... 6
3.2.2.1.4 The Constant Element ..... 6
3.2.2.2 Recap ..... 6
3.2.3 Struts2 in Action ..... 7
3.2.3.1 Create Action class ..... 7
3.2.3.1.1 Create Action class by extending ActionSupport ..... 7
3.2.3.1.1.1 Example code ..... 8
3.2.3.1.2 Create Action class using Action interface ..... 13
3.2.3.1.2.1 Method(s) in Action Interface ..... 13
3.2.3.1.2.2 Example ..... 13
3.2.3.1.3 Create Action class as POJO class ..... 14
3.2.3.2 Recap ..... 14
3.2.4 Struts2 Validations ..... 14
3.2.4.1 Using Annotations ..... 14
3.2.4.1.1 Example Scenario ..... 14
3.2.4.2 Using Validate Method ..... 21
3.2.4.2.1 Steps to Perform custom validation ..... 22
3.2.4.2.2 Example to perform custom validation ..... 23
3.2.4.3 Recap ..... 25

# COURSE 3STRUTS2 – PART III

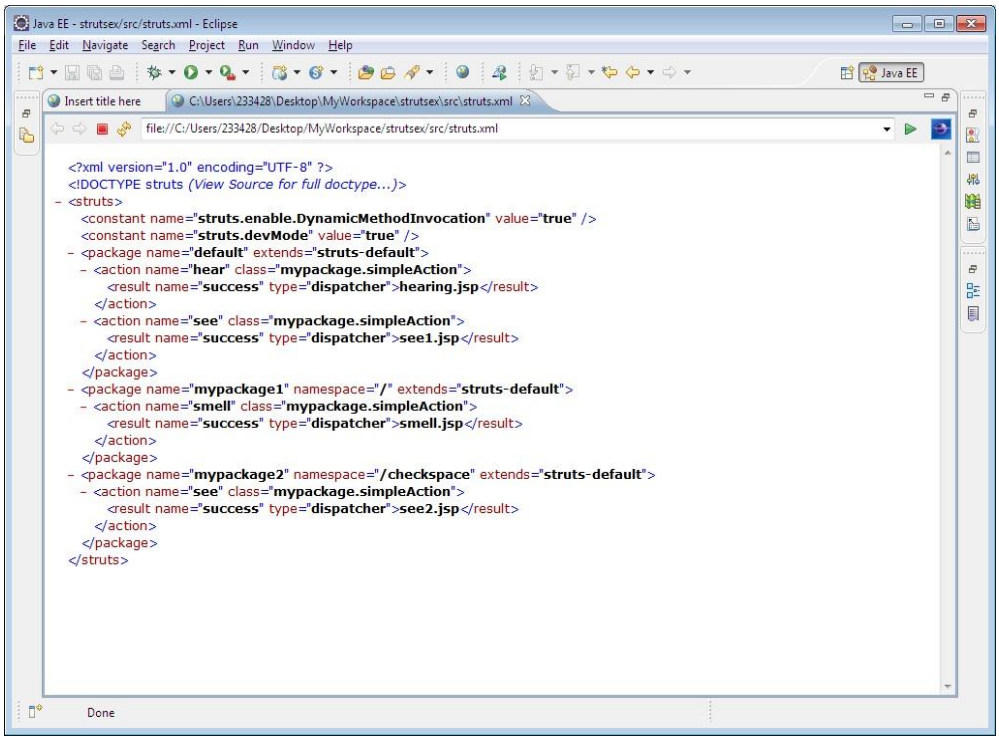
## 3.1 Objective

Objective of this document is to provide a detailed understanding about Struts 2.x Struts.xml configurations and action class.

## 3.2 Course Content

### 3.2.1 More about Struts.xml

The core configuration file for the Struts2 framework is the struts.xml file.



## 3.2.2 Exploring Struts.xml

The**<struts> tag** is the root tag for the **struts.xml.** The main child elements are as follows:

- Package
- Constant

### 3.2.2.1 More about Package Element

We can easily divide our struts application into sub modules. The package element specifies a module. You can have more packages in the struts.xml file.

Packages are a way to group actions, results, result types, interceptors, and interceptor-stacks into a logical configuration unit. Conceptually, packages are similar to objects in that they can be extended and have individual parts that can be overridden by "sub" packages.

Please refer to the picture below to see how package tag is defined in Struts.xml

```
<package name="default" namespace="/" extends="struts-default">
<action name="hear" class="mypackage.simpleAction">
<result name ="success" type="dispatcher">hearing.jsp</result>
</action>
<action name="see" class="mypackage.simpleAction">
<result name ="success" type="dispatcher">see1.jsp</result>
</action>
</package>
<package name="mypackage1" namespace="/" extends="struts-default">
<action name="smell" class="mypackage.simpleAction">
<result name ="success" type="dispatcher">smell.jsp</result>
</action>
</package>
<package name="mypackage2" namespace="/checkspace" extends="struts-default">
<action name="see" class="mypackage.simpleAction">
<result name ="success" type="dispatcher">see2.jsp</result>
</action>
</package>
```

#### 3.2.2.1.1 Attributes of Package Element

##### 3.2.2.1.1.1 name

*'name'* is a must for defining any package.

##### 3.2.2.1.1.2 Namespace

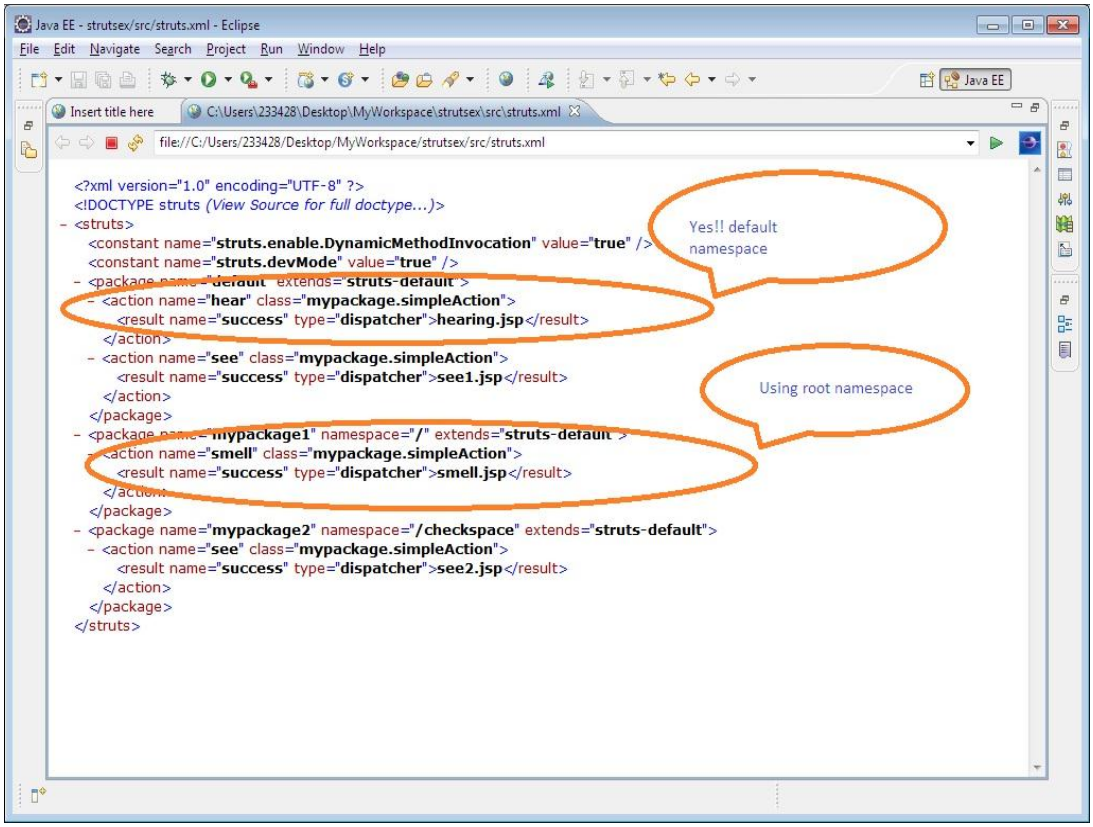
The '*namespace'* attribute subdivides action configurations into logical modules, each with its own identifying prefix. Namespaces avoid conflicts between action names.

###### 3.2.2.1.1.2.1 Default Namespace

The default namespace is "" - an empty string. The default namespace is used as a "catch-all" namespace. If an action configuration is not found in a specified namespace, the default namespace will be searched.

###### 3.2.2.1.1.2.2 Root Namespace

'*root'* namespace ("/") is also supported in Struts. The *'root'* is the namespace when a request directly under the context path is received. As with other namespaces, it will fall back to the default ("") namespace if a local action is not found.




**How does the code work?**

If a request for /checkspace/see.action is made, the /checkspace namespace is searched for the see action. If found, the see action is executed, else it will fall back to the default namespace. In the Namespace Example, the see action does exist in the /checkspace namespace, so the see action will be executed, and if "success" is returned, the request will be forwarded to see2.jsp.

If a request is made to /checkspace/hear.action, the namespace /checkspace will be checked for action hear. If a local action is not found, the default namespace is checked. In the Namespace Example, there is no action hear in the namespace /checkspace, therefore the default will be checked and /hear.action will be executed.

In the Namespace Example, if a request for smell.action is made, the root namespace ('/') is searched for a smell action; if a root action is not found, the default namespace is checked. In this case, the smell action does exist and will be executed. Upon success, the request would be forwarded to smell.jsp.

If a request is made for */hear* action, the root / namespace will be checked. If hear is found, the root action will be selected. Otherwise, the framework will check the default namespace. In the Namespace Example, the hear action does not exist in the root namespace, so the default namespace is checked, and the default hear action is executed.

- **extends -** The package element mostly extends the **struts-default** package where interceptors and result types are defined. If you extend struts-default, all the actions of this package can use the interceptors and result-types defined in the **struts-default.xml** file.

### 3.2.2.1.2 Action Element

The action is the sub element of package and represents an action.

#### 3.2.2.1.2.1 Attributes of action element

- **name** name is must for defining any action.

- **class** class is the optional attribute of action. If you omit the class attribute, **ActionSupport** will be considered as the default action. A simple action may be as:

  <action name="product">

- **method** It is an optional attribute. If you don't specify method attribute, **execute** method will be considered as the method of action class. So this code:

  <action name="product" class="com.ilp.tcs.com.Product">

This will be same as:

<action name="product" class="com.ilp.tcs.com.Product" method="execute">

If you want to invoke a particular method of the action, you need to use method attribute.

### 3.2.2.1.3 Result Element

When an Action class method completes, it returns a String. The value of the String is used to select a result element. An action mapping will often have a set of results representing different possible outcomes. A standard set of result tokens are defined by the ActionSupport base class.

```
String SUCCESS = "success";
String NONE = "none";
String ERROR = "error";
    String INPUT = "input";
```

#### 3.2.2.1.3.1 Attributes of result element

- **name** is the optional attribute. If you omit the name attribute, success is assumed as the default result name.

- **type** is the optional attribute. If a type attribute is not specified, the framework will use the default dispatcher type, which forwards to another web resource. If the resource is a Java Server Page, then the container will render it, using its JSP engine.

```
<result type="freemarker">foo.fm</result>
```

- The param tag sets a property on the Result object. The most commonly-set property is location, which usually specifies the path to a web resources.

Illustration: without param tag

```
<result name="success"> /HelloWorld.jsp</result>
```

Illustration: with param tag

```
<result name="success" type="dispatcher">
```

```
  <param name="location">
     /HelloWorld.jsp
  </param >
</result>
```

### 3.2.2.1.3.2 Global Results

Most often, results are nested with the action element. But some results apply to multiple actions. In a secure application, a client might try to access a page without being authorized, and many actions may need access to a "logon" result.

If actions need to share results, a set of global results can be defined for each package. The framework will first look for a local result nested in the action. If a local match is not found, then the global results are checked.

```
<struts>
      <package name="login" namespace="/" extends="struts-default">

       <global-results>
       <result name="checkexception">globalresult.jsp</result>
       </global-results>
```

### 3.2.2.1.4 The Constant Element

There are two key roles for constants.

They are used to override struts basic settings like the maximum file upload size or to define whether the application is in development mode etc.

In the various XML variants, the constant element has two required attributes: name and value.

| Attribute | Required | Description |
|---|---|---|
| name | **yes** | the name of the constant |
| value | **yes** | the value of the constant |

```
<struts>

    <constant name="struts.enable.DynamicMethodInvocation" value="true" />
    <constant name="struts.devMode" value="true" />
```

### 3.2.3 Recap

In this section you have seen

- what is the role of struts.xml in a Struts application
- main tags in struts.xml and their main attributes
- how to use the different tags in struts.xml

### 3.2.4 Struts2 in Action

The action classes are core concepts in Struts2; it is Model class itself and contains the business logic for processing on the data. Action classes act as controller in the MVC (Model View Controller) pattern. So actions are the core part of Struts2 framework. The main functionalities of action classes is respond to a user request, execute business logic, and then return a result to the user based on configuration file (struts.xml) to render the view page.

Each URL is mapped to a specific action, which provides the processing logic necessary to service the request from the user.
But the action also serves in two other important capacities. First, the action plays an important role in the transfer of data from the request through to the view, whether its a JSP or other type of result. Second, the action must assist the framework in determining which result should render the view that will be returned in the response to the request.

Stuts2 actions are not singleton, means instance of action is created on each request. Hence Struts2 action need not be thread safe like Struts1 actions. The action classes are simple and do not dependent on other API. So testing of action classes is much easier.

#### 3.2.4.1 Create Action class

The only requirement for actions in Struts2 is that there must be one no-argument method that returns either a String or Result object and must be a POJO. If the no-argument method is not

specified, the default behaviour is to use the execute() method.
The simple action class may look like:

**Welcome.java**

```
package com.javaex;
  public class Welcome {
  public String execute(){
   return "success";
} }
```

There are three ways to create actions in Struts2 :

### 3.2.4.1.1 Create Action class by extending ActionSupport

Action classes usually extend the ActionSupport class, which is provided by the Struts 2 framework and provides most common functionality of web application like ; validate data, action level message, action level error message, field level error message and read resource bundled

#### 3.2.4.1.1.1 Example code

To illustrate the point that the action method controls the view, let us make the following change to the **execute()** method and extend the class ActionSupport as follows:

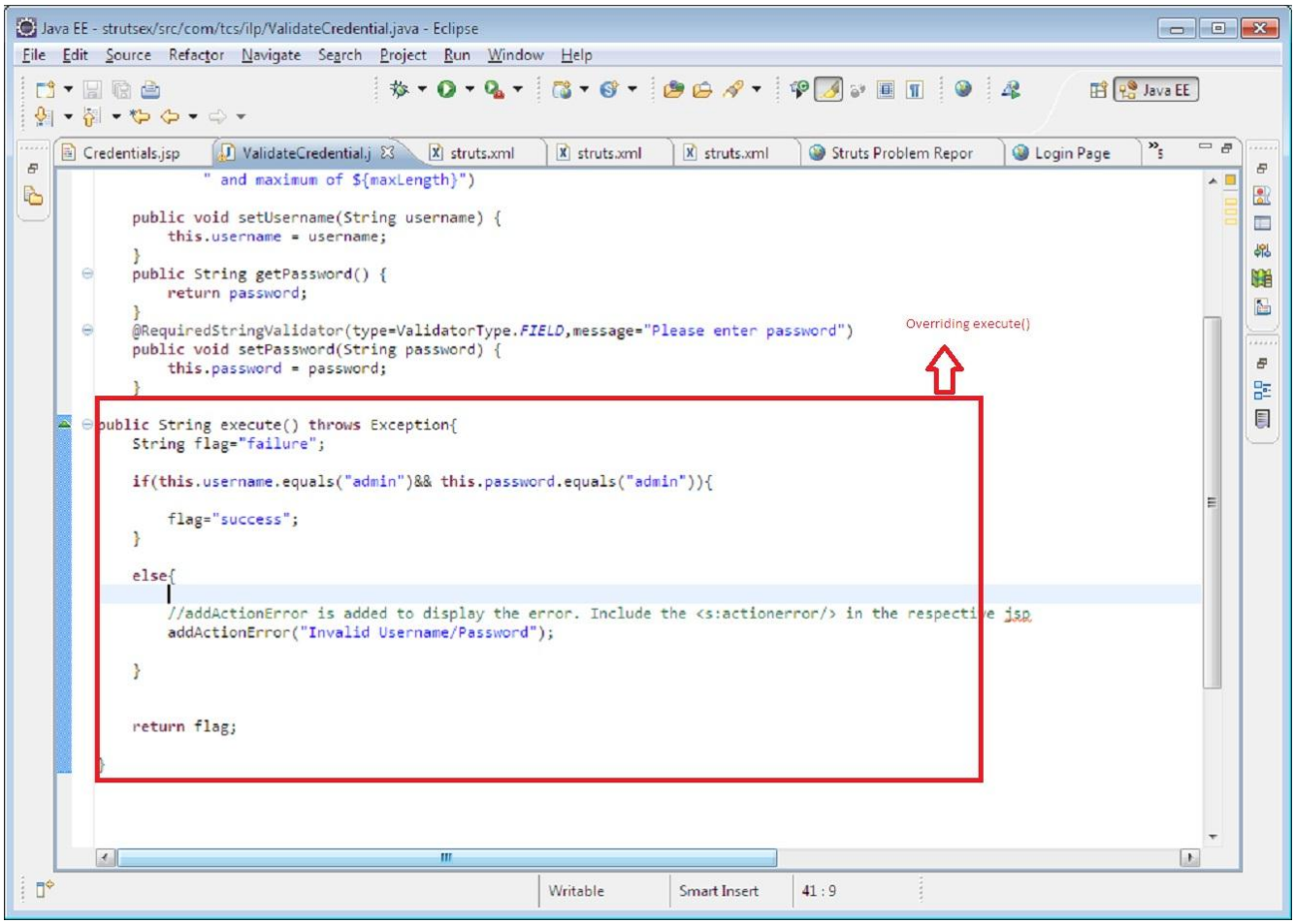
Java EE - strutsex/src/com/tcs/ilp/ValidateCredential.java - Eclipse

File Edit Source Refactor Navigate Search Project Run Window Help

Credentials.jsp | ValidateCredential.j | struts.xml | struts.xml | struts.xml | Struts Problem Repor | Login Page

```
package com.tcs.ilp;
import com.opensymphony.xwork2.ActionSupport;
@Validation
public class ValidateCredential extends ActionSupport {
    private String username;
    private String password;

    public String getUsername() {
        return username;
    }
    @RequiredStringValidator(type=ValidatorType.FIELD,message="Please enter username")
    @StringLengthFieldValidator(maxLength="9",minLength="2",trim=true,message="Username should be of minimun ${minLength}" +
            " and maximum of ${maxLength}")

    public void setUsername(String username) {
        this.username = username;
    }
    public String getPassword() {
        return password;
    }
    @RequiredStringValidator(type=ValidatorType.FIELD,message="Please enter password")
    public void setPassword(String password) {
        this.password = password;
    }

public String execute() throws Exception{
    String flag="failure";

    if(this.username.equals("admin")&& this.password.equals("admin")){

        flag="success";
    }

    else{

        //addActionError is added to display the error. Include the <s:actionerror/> in the respective jsp
```

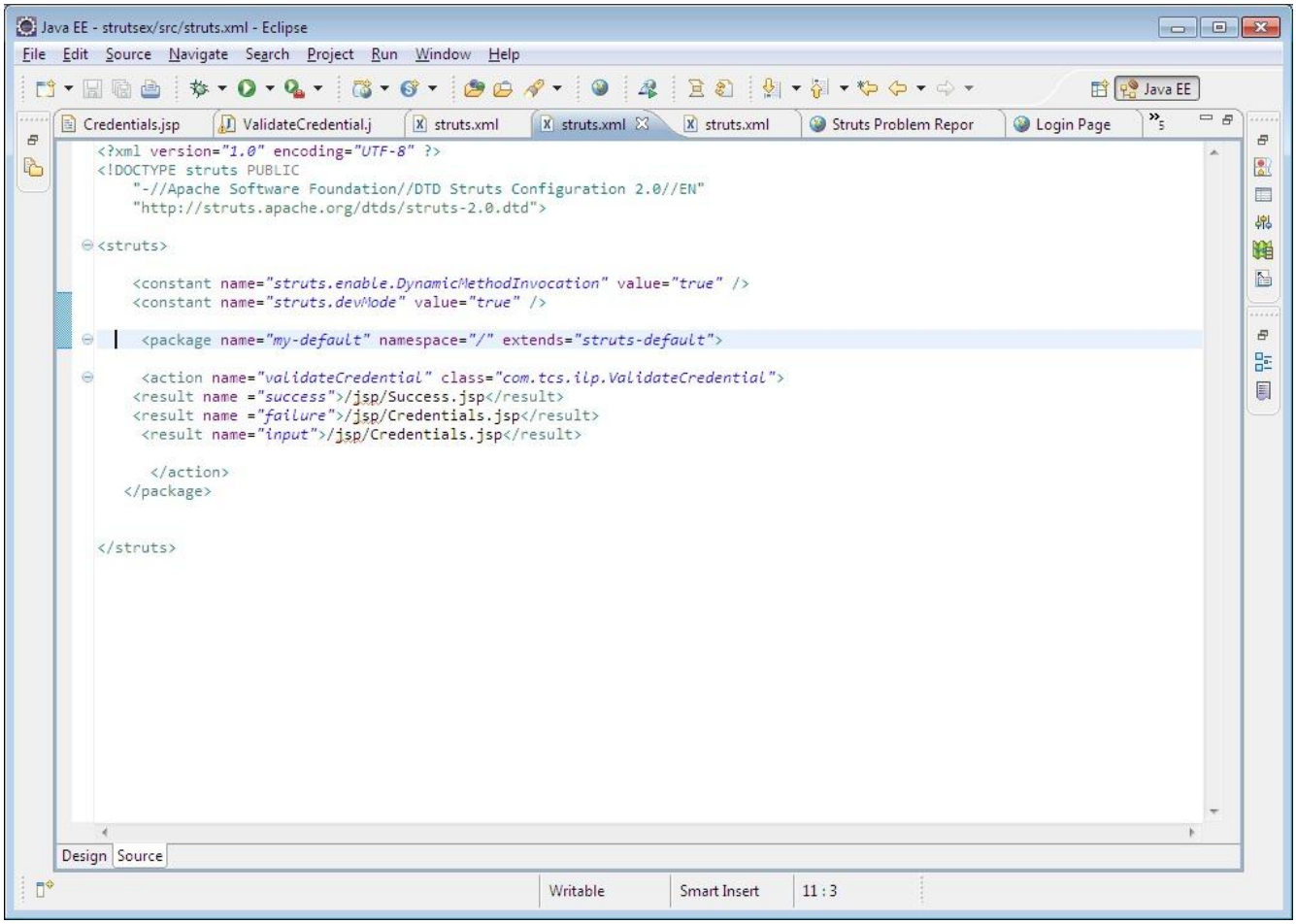
Writable | Smart Insert | 32 : 7




In this example, simple business logic is implemented in execute method i.e., if the credentials are correct, we return SUCCESS as the result as we have extended ActionSupport, we can use String constants SUCCESS.

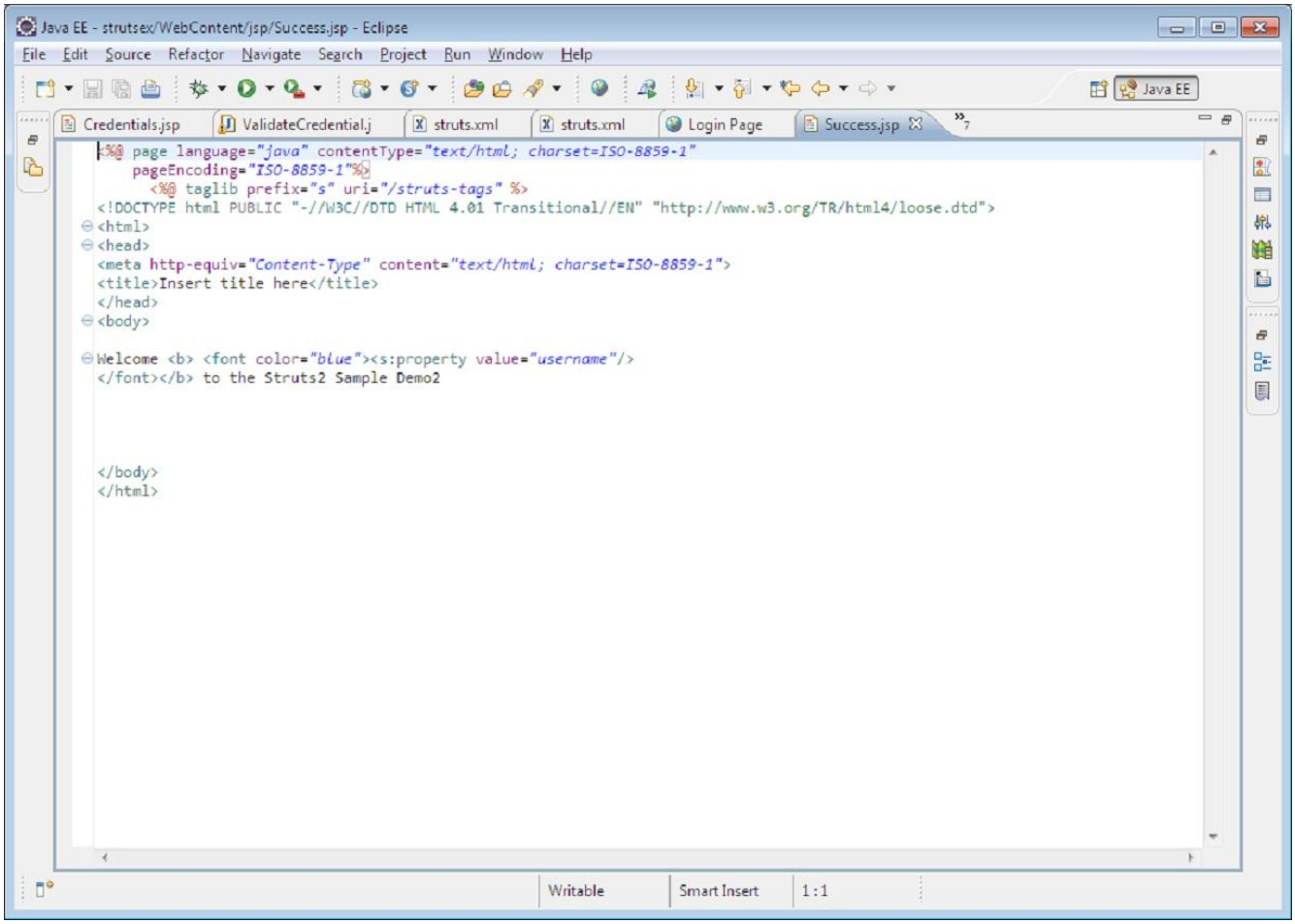
In this example, we have some logic in the execute method .If the credentials are correct, we return SUCCESS as the result. As we have extended ActionSupport, we can use String constants SUCCESS.

Now, let us modify our struts.xml file as follows:



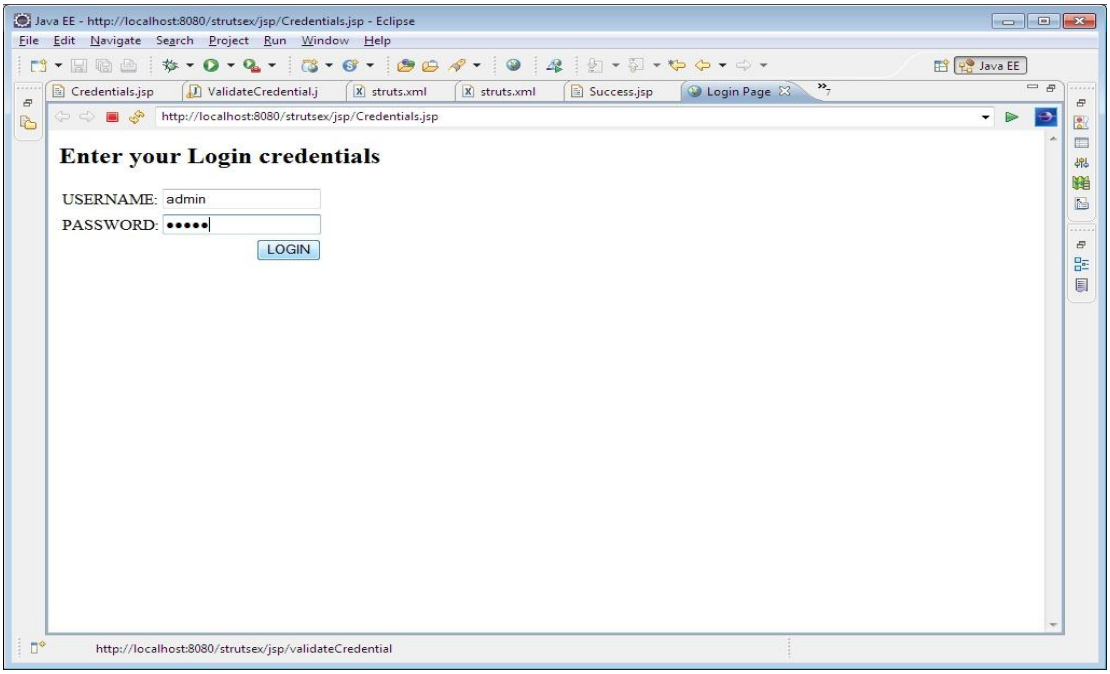

Success.jsp

This file will be called in case return result is SUCCESS which is a String constant "success" as defined in Action interface:

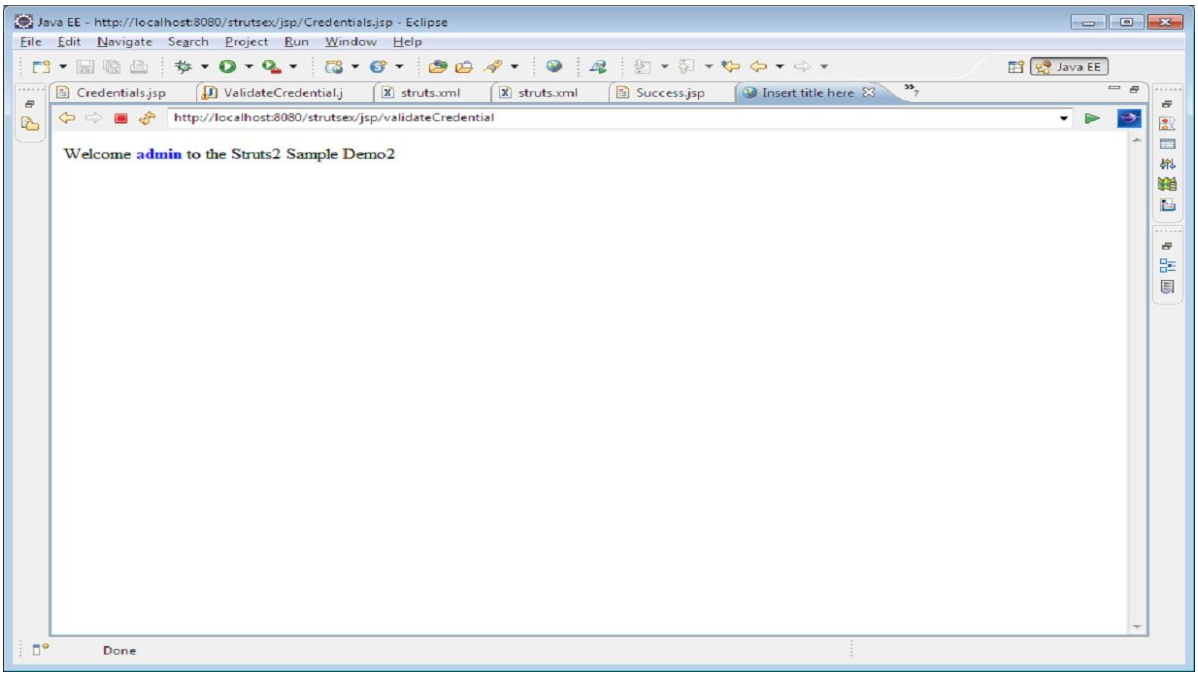



Run the application and see the output:

Enter the login credentials




See the output on successful login




All Struts2 actions may implement this interface, which exposes the execute() method.

**com.opensymphony.xwork2.Action** interface that defines 5 constants and one execute method.

### 3.2.4.1.2 Create Action class using Action interface

Action interface provides 5 constants that can be returned form the action class. They are:

1. **SUCCESS** indicates that action execution is successful and a success result should be shown to the user.
2. **ERROR** indicates that action execution is failed and a error result should be shown to the user.
3. **LOGIN** indicates that user is not logged-in and a login result should be shown to the user.
4. **INPUT** indicates that validation is failed and a input result should be shown to the user again.
5. **NONE** indicates that action execution is successful but no result should be shown to the user.

Let's see what values are assigned to these constants:

```
public interface Action {
    public static final String SUCCESS = "success";
    public static final String NONE = "none";
    public static final String ERROR = "error";
    public static final String INPUT = "input";
    public static final String LOGIN = "login";
    public String execute() throws Exception;
}
```

#### 3.2.4.1.2.1 Method(s) in Action Interface

Action interface contains only one method execute that should be implemented overridden by the action class even if you are not forced.

```
public String execute();
```

#### 3.2.4.1.2.2 Example

If we implement the Action interface, we can directly use the constants instead of values.

**Welcome.java**

```
package com.javaex;
import com.opensymphony.xwork2.Action;
public class Welcome implements Action{
public String execute(){
    return SUCCESS;
}
}
```

#### 3.2.4.1.3 Create Action class as POJO class

The Struts2 also supports action class as POJO. You are free to define any entry level method other than execute(), but you have to configure it in struts.xml file. By default Struts2 framework call execute() method from POJO action class.

### 3.2.4.2 Recap

- In this section you have seen,
- What are action classes
- How to create an Action class by
  - extending Action Support class
  - using Action interface
  - as a POJO class

## 3.2.5 Struts2 Validations

### 3.2.5.1 Using Annotations

#### 3.2.5.1.1 Example Scenario

**Scenario Description:** Develop login form

**Step 1**: Create the JSP file as given below.

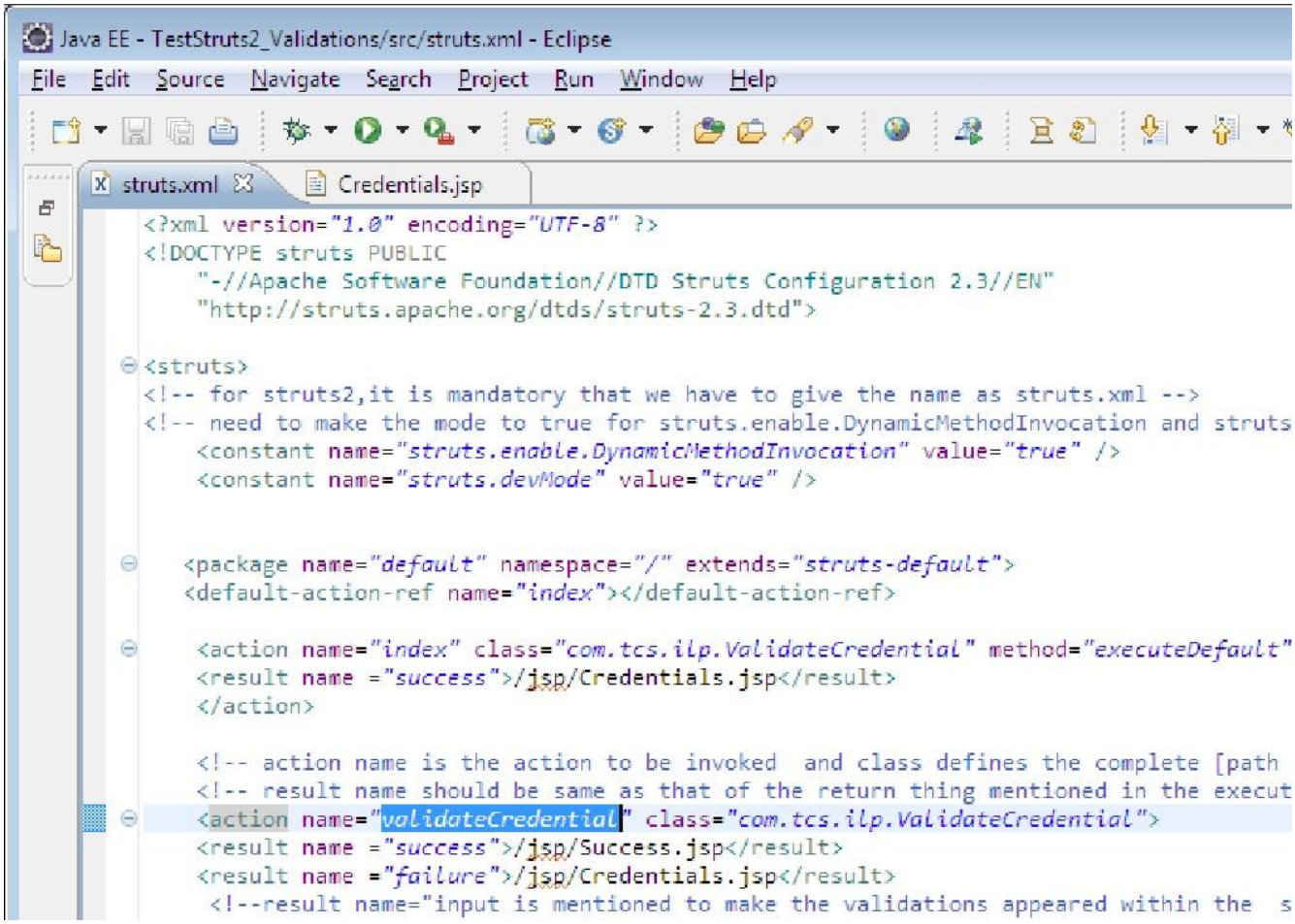
Java EE - TestStruts2_Validations/WebContent/jsp/Credentials.jsp - Eclipse

File Edit Source Refactor Navigate Search Project Run Window Help

struts.xml | Credentials.jsp

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
    <%@ taglib prefix="s" uri="/struts-tags" %>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/ht
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Login Page</title>
</head>
<body>
<h2>Enter your Login credentials</h2>
<!--actionerror cssStyle is used to make the errors (validations) within the form to be
<s:actionerror cssStyle="color:red"/>
<s:form action="validateCredential">
<s:textfield label="USERNAME" name="username"></s:textfield>

<s:textfield label="PASSWORD" name="password" type="password"></s:textfield>

<s:submit value="LOGIN" ></s:submit>
</s:form>

</body>
</html>
```

**Step 2: Configure action mapping in struts.xml**




**Step 3: Create Action class**

Now let's develop the action class to handle the login request.

The Struts 2 framework provides a base ActionSupport class to implement commonly used interfaces. In our action class (AnnotationAction.java) we have extended ActionSupport class and imported the com.opensymphony.xwork2.validator.annotations package .
For validating the login application Javascript can be added to the jsp page or in action class, but **Struts 2 provides** another very easy method to validate your form fields using annotations in the action class.
Two annotations are needed.

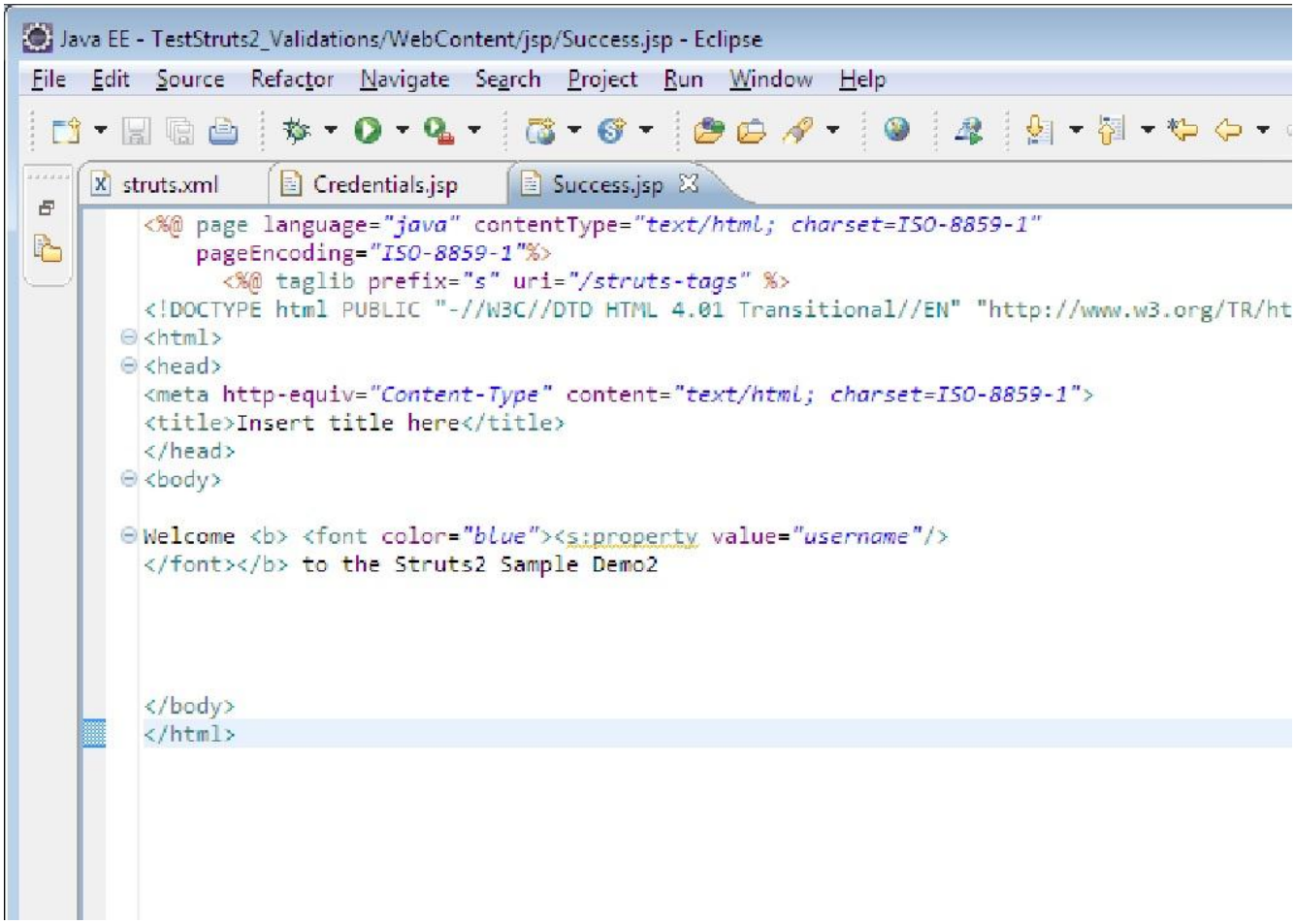
1. The **@Validation** annotation tells to Struts that **action in this class might need to be validated**.
2. The **@RequiredStringValidator** annotation is used for the **text input to hold a singular value**.

Java EE - TestStruts2_Validations/src/com/tcs/ilp/ValidateCredential.java - Eclipse

File Edit Source Refactor Navigate Search Project Run Window Help

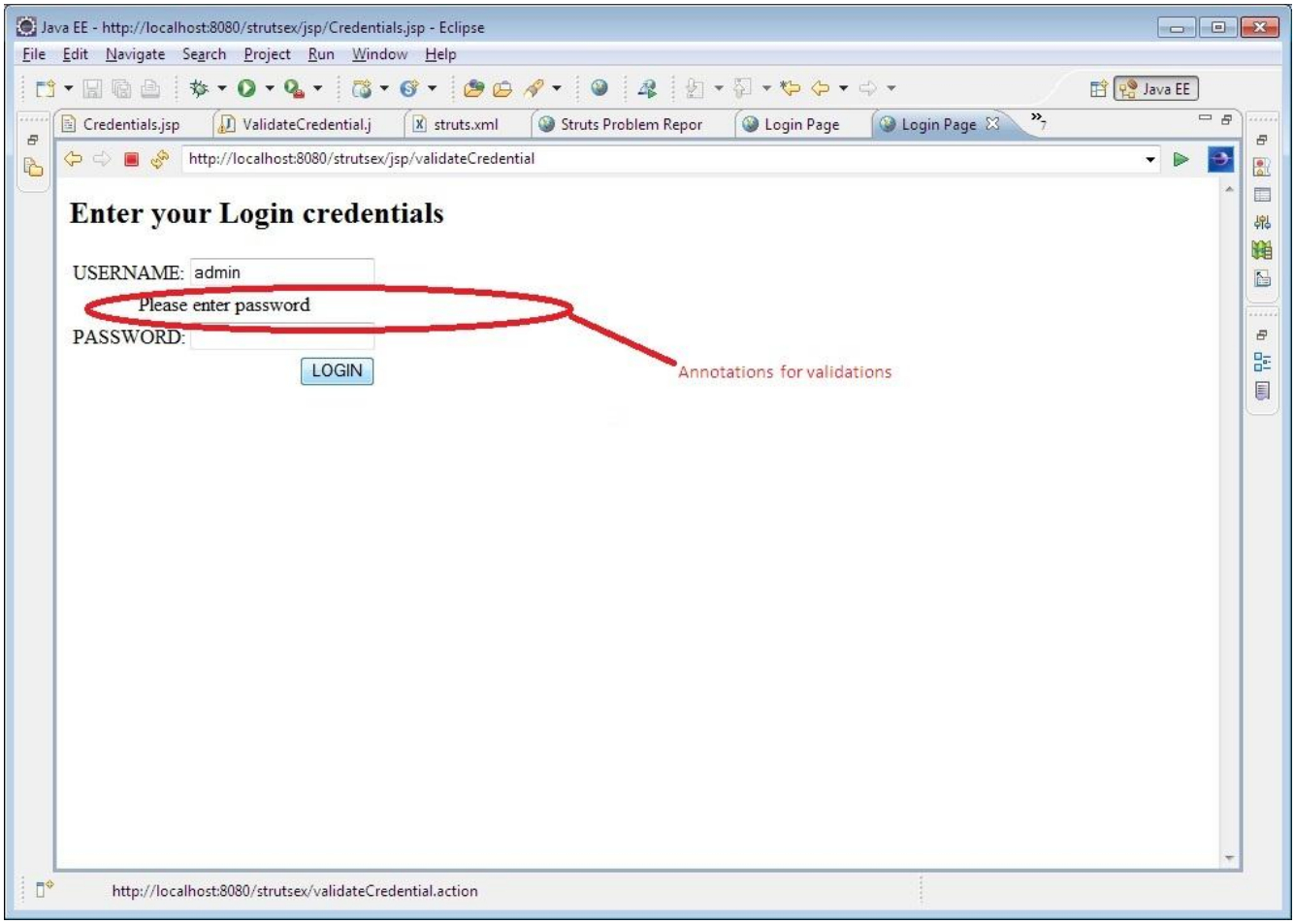
struts.xml | Credentials.jsp | ValidateCredential.java

```
package com.tcs.ilp;
import com.opensymphony.xwork2.ActionSupport;
@Validation
public class ValidateCredential extends ActionSupport {
    private String username;
    private String password;
    private String message;
    public String getUsername() {
        return username;
    }
    @RequiredStringValidator(type=ValidatorType.FIELD,message="Please enter username")
    @StringLengthFieldValidator(maxLength="9",minLength="2",trim=true,message="Username s
            " and maximum of ${maxLength}")

    public void setUsername(String username) {
        this.username = username;
    }
    public String getPassword() {
        return password;
    }
    @RequiredStringValidator(type=ValidatorType.FIELD,message="Please enter password")
    public void setPassword(String password) {
        this.password = password;
    }
```

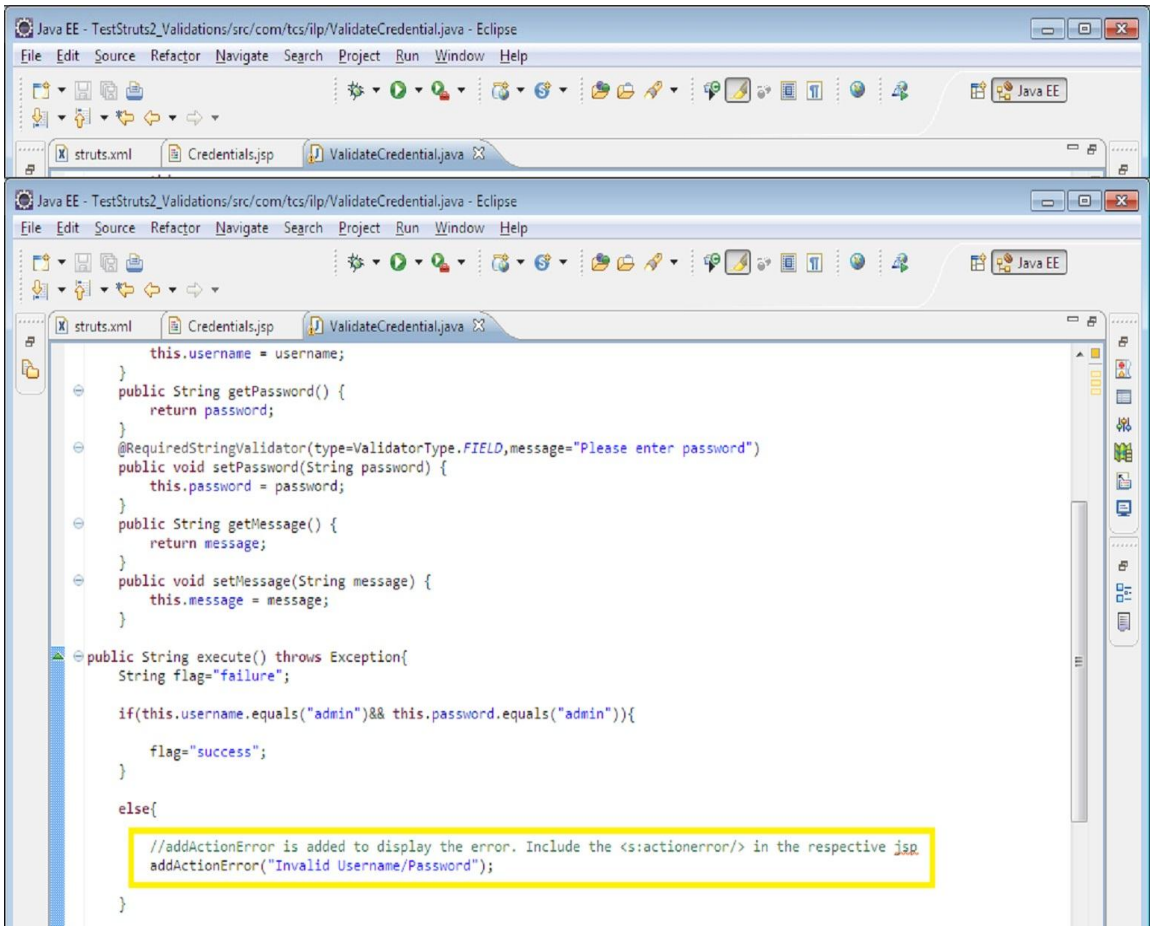


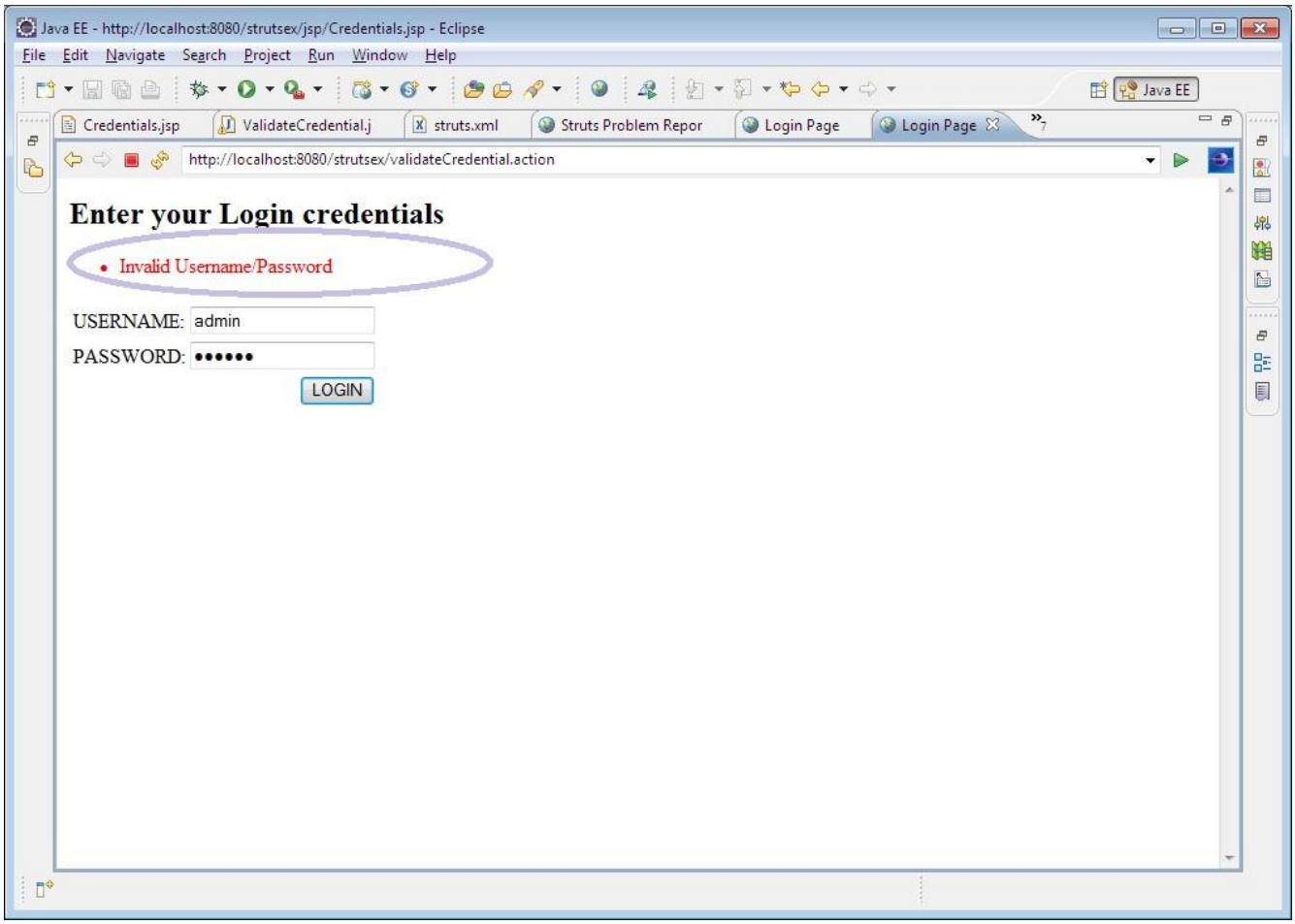

**Step 4: Run the application and see the output**

**I**f you fill only the "USERNAME" field and click LOGIN button without filling the PASSWORD field, the output is:


Java EE - http://localhost:8080/strutsex/jsp/Credentials.jsp - Eclipse
File Edit Navigate Search Project Run Window Help
Java EE
Credentials.jsp
ValidateCredential.j
struts.xml
Struts Problem Repor
Login Page
Login Page
http://localhost:8080/strutsex/jsp/validateCredential
Enter your Login credentials
USERNAME: admin
Please enter password
PASSWORD:
LOGIN
Annotations for validations
http://localhost:8080/strutsex/validateCredential.action

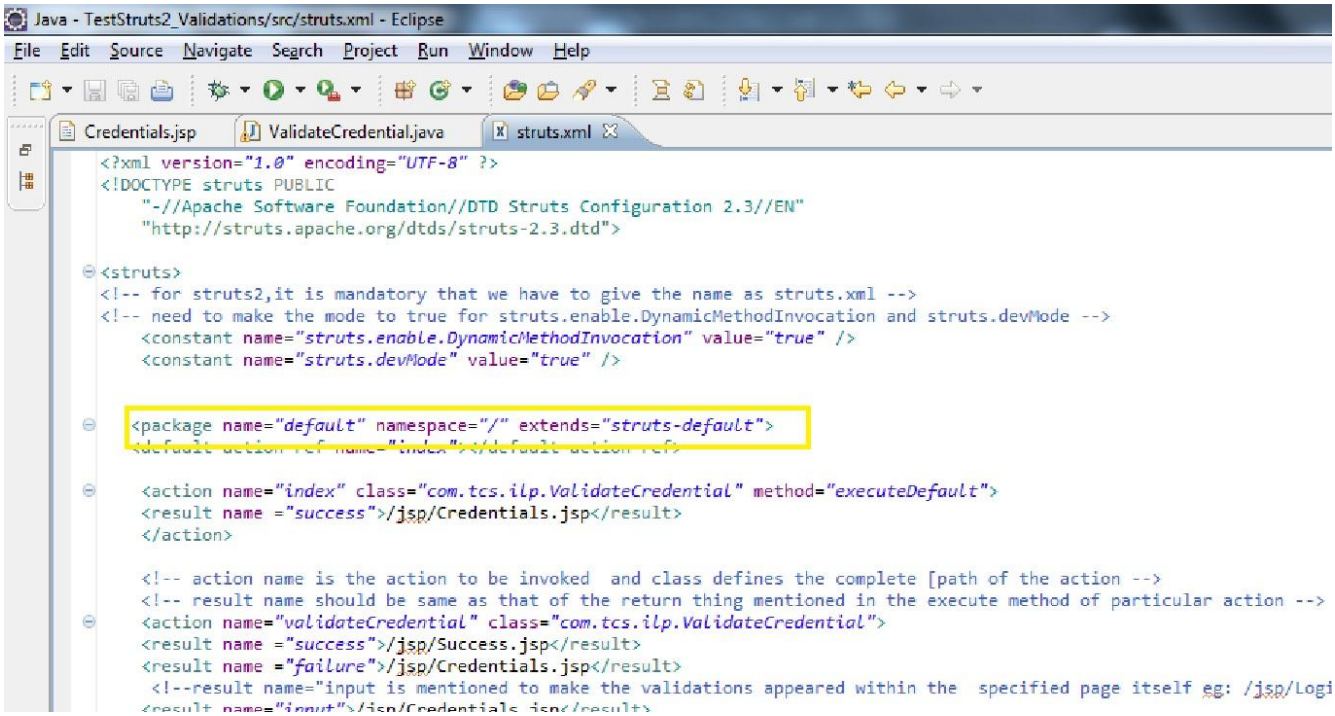
Now, how will you show the error if you fill wrong credentials and click the Login button**?**
You use **addActionError()** .Include the tag **<s:actionerror/>** in the respective jsp where you want the error to be displayed.






### 3.2.5.2 Using Validate Method

Let's see sample struts.xml




Our "default" package extends "struts-default" package. By extending the "struts-default" package the action will by default inherit the set of interceptors defined in the default stack. The "struts-default" package is defined in the struts-defaulter file.

Common tasks like validation, logging etc separated out and are placed in different interceptors. You can define an interceptor stack for each action. Most commonly used interceptors are grouped in default stack of the struts-default package. The default stack will be sufficient in most cases. The interceptors will be fired in the order in which they are declared in the stack both before and after the action is executed.
The workflow interceptor controls the flow of control.
The workflow interceptor checks whether the action implements the Validateable interface, if it does, the workflow interceptor will invoke the validate() method of the Action class.
In the validate() method we validate the user name and the password. If the validation fails an error is added using the addFieldError() method.
The validate() method doesn't return any errors, instead it stores all the errors with the help of the ValidationAware interface.
Now the workflow interceptor will check if any validation errors have occurred. If any error has occurred the work flow interceptor will stop the request processing and transfer the control to the input page with appropriate error messages.

### 3.2.5.2.1 Steps to Perform custom validation

The steps are as follows:

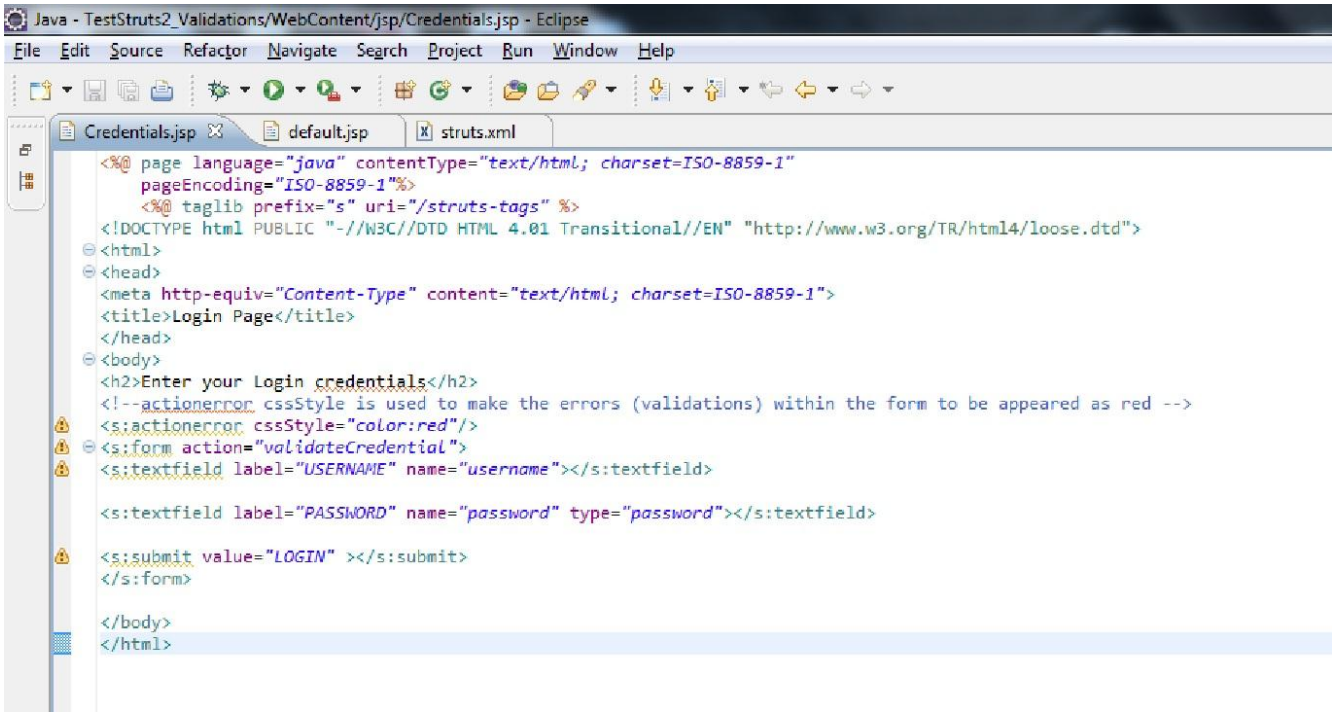
1. create the form to get input from the user
2. Define the validation logic in action class by extending the ActionSupport class and overriding the validate method
3. Define result for the error message by the name input in struts.xml file

### 3.2.5.2.2 Example to perform custom validation

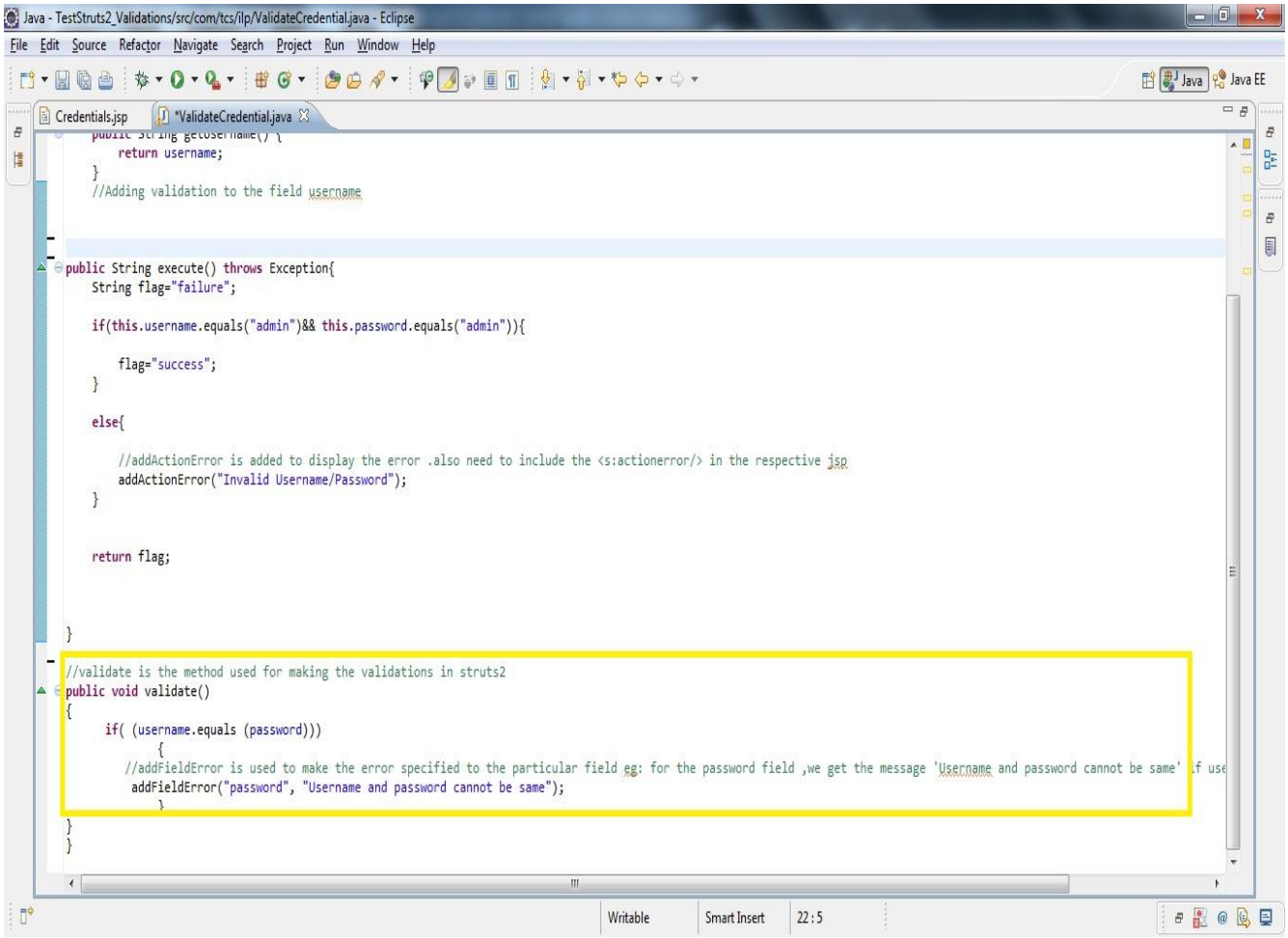
In this example, we are creating 4 pages:

1. Credential for input from the user.
2. ValidateCredential.java for defining the validation logic.
3. struts.xml for defining the result and action.
4. Success.jsp for the view component.

**Step1:** Create **Credentials.jsp** for input
This jsp page creates a form using struts UI tags. It receives name and password from the user.




**Step2:** Create the **action** class
This action class inherits the ActionSupport class and overrides the validate method to define the validation logic.




**Step3: Define a input result in struts.xml**

This XML file defines an extra result by the name input, that will be invoked if any error message is found in the action class.

Java - TestStruts2_Validations/src/struts.xml - Eclipse

File Edit Source Navigate Search Project Run Window Help

Credentials.jsp | ValidateCredential.java | struts.xml

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE struts PUBLIC
    "-//Apache Software Foundation//DTD Struts Configuration 2.3//EN"
    "http://struts.apache.org/dtds/struts-2.3.dtd">

<struts>
<!-- for struts2,it is mandatory that we have to give the name as struts.xml -->
<!-- need to make the mode to true for struts.enable.DynamicMethodInvocation and struts.devMode -->
    <constant name="struts.enable.DynamicMethodInvocation" value="true" />
    <constant name="struts.devMode" value="true" />


   <package name="default" namespace="/" extends="struts-default">
   <default-action-ref name="index"></default-action-ref>

    <action name="index" class="com.tcs.ilp.ValidateCredential" method="executeDefault">
    <result name ="success">/jsp/Credentials.jsp</result>
    </action>

    <!-- action name is the action to be invoked  and class defines the complete [path of the action -->
    <!-- result name should be same as that of the return thing mentioned in the execute method of particular action -->
    <action name="validateCredential" class="com.tcs.ilp.ValidateCredential">
    <result name ="success">/jsp/Success.jsp</result>
    <result name ="failure">/jsp/Credentials.jsp</result>
     <!--result name="input is mentioned to make the validations appeared within the  specified page itself eg: /jsp/Login.jsp-->
    <result name="input">/jsp/Credentials.jsp</result>

    </action>

    </package>
```

**Step4: Run the application and see the output**

## 3.2.5.3 Recap

From the previous sections you have understood the different ways of doing validation using Struts

- Using annotations
- using validate method